Reliable Third-Party Library Detection in Android and itsSecurity Applications

1. What is it?

This paper introduces a tool called **LibScout**. It's designed to find Third-Party Libraries (TPLs) inside Android APKs. The main goal of the tool is to be very accurate, even when the code is obfuscated, and to find the exact version of the library instead of just the name.

2. Who wrote it and who is it for?

It was written by researchers from **CISPA (Saarland University)**, including Michael Backes and Erik Derr. It's mainly for people doing security audits. It helps them understand how long it takes for a developer to fix a vulnerability in their app after a library update is released.

3. Why was it created? (The Problem)

The authors noticed that developers are very slow at updating libraries, which leaves users at risk. Most tools in 2016 couldn't handle **Identifier Renaming** (obfuscation), where package names like `com.google` are changed to things like `a.b.c`. Also, just knowing a library is there isn't enough for security; you need the exact version number to check against a **CVE database**. If you don't have the version, you can't be sure if the app is actually vulnerable.

4. How does it work? (The Methodology)

Instead of looking at names or raw code (which can be changed easily), LibScout looks at the **structure** of the library.

- **Merkle Trees:** They build a 3-layer hash tree (Package -> Class -> Method). This makes the signature very organized.
- **Fuzzy Descriptors:** This is a very important part. They remove the method names and only keep the "descriptor" (the types of data going in and out). If it's a custom type, they replace it with an "X".
    - *Example:* `open(Activity, bool)` becomes `(android.app.Activity, bool, X)X`.
    - This works because obfuscators usually don't change the types of arguments, only the names.
- **CHA (Class Hierarchy Analysis):** It looks at how the classes are linked together—the "skeleton"—which stays the same even if the logic inside is slightly different.
- **Similarity Score:** Since some apps remove parts of a library to save space (dead-code elimination), the tool gives a score from 0 to 1. If it's 1.0, it's a perfect match.

5. Key Findings of the Paper

The researchers found that the **"Vulnerability Window"** is quite large. For example, it took about **190 days** on average for apps to update to a safe version of the Facebook SDK. They

also found that nearly **3.7 billion device installs** were affected by libraries using bad cryptographic practices. Technically, they proved that LibScout is 100% resistant to basic renaming obfuscation.

6. Impact on Project

For my project, this paper is very useful because:

- It gives me a clear way to build a **signature database**. I can use their idea of a 3-layer hierarchy for my own tool.
- It justifies why I should use **Fuzzy Descriptors** to handle obfuscation instead of just searching for strings.
- It shows that I need to build my "ground truth" using original **Library SDKs** to make sure the detection is reliable. I shouldn't just guess based on other apps; I need to use the real files from Maven or other repositories.

Automated Third-Party Library Detection for Android Applications: Are We There Yet?

1. What is it?

This paper, titled **"Automated Third-Party Library Detection for Android Applications: Are We There Yet?"**, is a large-scale empirical study. Instead of introducing just one new tool, the authors take all the existing tools (like LibScout, LibRadar, and LibPecker) and test them against each other to see which one actually works best in real-world conditions.

2. Who wrote it and who is it for?

- **Authors:** A team of researchers from several universities, including **The Hong Kong Polytechnic University** and **Nanyang Technological University**.
- **Audience:** It is for researchers and developers who need to know which tool to choose for specific tasks, such as finding malware, detecting vulnerable libraries, or analyzing apps at a very large scale.

3. Why was it created? (The Problem)

The authors noticed that while many tools have been built to find third-party libraries (TPLs), nobody really knew which one was the most effective. TPLs make up over **60% of the code** in most apps, but they also bring security risks like malicious code or privacy leaks. The "problem" was that there was no fair, unified way to compare these tools—each tool used different datasets and different metrics, making it impossible to tell which one was truly reliable against modern obfuscation.

4. How does it work? (The Methodology)

The researchers conducted a massive experiment using four main criteria:

- **Effectiveness:** How many libraries does the tool find correctly (Precision and Recall)?
- **Efficiency:** How fast is the tool when scanning thousands of apps?
- **Obfuscation-resilience:** Can the tool still find the library if the developer renames the packages or hides the code?
- **Ease of Use:** Is the tool actually easy for a human to install and understand the results?

They built a "benchmark" dataset of **59 unique TPLs** with over **2,000 versions** and tested five major open-source tools (LibID, LibRadar, LibScout, LibPecker, and ORLIS).

5. Key Findings of the Paper

The paper provides a "winner" for different categories:

- **Best Overall Accuracy: LibScout** was the most effective at identifying specific library versions.
- **Best Speed: LibRadar** was the fastest (average of 5.53 seconds per app) and the easiest to use.
- **Best Against Obfuscation: LibPecker** performed best when the code was heavily scrambled.
- **The "Multidex" Issue:** They found that many tools fail because they only look at the main code file (`classes.dex`) and ignore the extra files in large apps, which causes them to miss about half of the libraries.

6. Impact on Project

This paper is extremely helpful for our project because it acts as a **"Buyer's Guide"** for detection techniques:

- **Tool Selection:** If our project focuses on **security**, we should use the logic from **LibScout**. If we need to scan **millions of apps** quickly, we should follow **LibRadar's** API-based approach.
- **Handling Large Apps:** It taught me that we *must* support **"Multidex"** (scanning all .dex files), or our results will be 50% lower than they should be.
- **Testing:** We can use their publicly available dataset to test our own implementation to see if it is actually "obfuscation-resilient".

Libd: Scalable and precise third-party library detection in android markets

1. What is it?

This paper introduces **LibD**, a tool designed to find third-party libraries (TPLs) in a way that is both scalable and highly accurate. It focuses on solving the problem of "multi-package" libraries—where one library is spread across several different folders—and ensures the detection works even if the developer has used obfuscation to hide the code.

2. Who wrote it and who is it for?

- **Authors:** A group of researchers from the **Chinese Academy of Sciences** and **Pennsylvania State University**, led by Menghao Li.
- **Audience:** It is mainly for security researchers who need to analyze massive datasets (millions of apps) to find vulnerable or malicious libraries that might have been repackaged by attackers.

3. Why was it created? (The Problem)

The authors found two major flaws in older tools like LibRadar:

- **The Package Name Trap:** Many tools assume that a library will always have the same package name (like `com.facebook`). But if an app is obfuscated, these names change to things like `a.b.c`, and the tools fail.
- **The Structure Problem:** Older tools assume a library is just one "subtree" or folder. In reality, a library can have different internal structures or be split into multiple parts across different versions.
- **Security Risks:** The authors wanted to stop attackers from tampering with ad libraries or social media SDKs to steal revenue or user data.

4. How does it work? (The Methodology)

LibD doesn't just look at names; it looks at how the code is "wired" together:

- **Library Instance Recovery:** It uses **Inclusion, Inheritance, and Call relations** to find the boundaries of a library. It builds a "Homogeny Graph" to see which packages are actually part of the same functional unit.
- **Feature Generation (Hashing):**
    - **Method Level:** It builds a Control Flow Graph (CFG) for every method and hashes the **opcodes** (the actual instructions).
    - **Class Level:** It combines these method hashes into a class hash.
    - **Instance Level:** Finally, it creates a unique "fingerprint" for the whole library.
- **Obfuscation Resilience:** Because it hashes the *instructions* (opcodes) rather than the *names*, it doesn't matter if the developer renames every class and method—the hash remains the same.

5. Key Findings of the Paper

- **Massive Scale:** They tested LibD on over **1.4 million apps**.
- **Outperforming Others:** LibD found **1,954 libraries** in a test set where LibRadar only found 670. It had a much lower "false negative" rate, meaning it missed far fewer libraries.
- **Common Mutations:** They found that libraries change constantly. For example, the `google/gson` library had **421 different mutations** (slight code changes) across the apps they scanned.
- **Ghost Calls:** They discovered "ghost calls"—code that calls functions that don't even exist in the app—which often points to hidden malicious behavior or poor reverse engineering.

6. Impact on Project

For our project, LibD teaches us a very important lesson about **boundaries**:

1. **Don't trust folder structures:** We can't assume one folder = one library. We need to look at the "call relations" (who is calling whom) to see the true size of a library.
2. **Opcode Hashing:** This is the best way to handle modern obfuscation. If we use this method, our tool will be much harder to "fool" than a tool that only looks at API names.
3. **Threshold Setting:** The paper suggests that a library should appear in at least **10 different apps** to be considered a "real" library. We can use this same threshold for our own database to filter out "noise" or app-specific code.

Pay Your Attention on Lib! Android Third-Party Library Detection via Feature Language Model (2025)

1. What is it?

This paper presents **LibAttention**, the first tool to use a **Feature Language Model** (specifically a Transformer/GPT-like architecture) to detect third-party libraries in Android apps. It moves away from traditional "if-then" rules and uses deep learning to "understand" what library code looks like, even if it has been heavily modified.

2. Who wrote it and who is it for?

- **Authors:** A research team from **Shanghai Jiao Tong University**, led by Dahan Pan and Yuanyuan Zhang.
- **Audience:** Security researchers who are struggling with the **Android R8 compiler**. Older tools were built for ProGuard, but R8 is much more aggressive, and this paper is for those who need a tool that can "see through" R8's optimizations.

3. Why was it created? (The Problem)

The main "villain" in this paper is the **Android R8 compiler**.

- **The R8 Failure:** The authors found that state-of-the-art tools like LibScan have an **F1-score of only 1%** when facing R8-obfuscated apps because R8 doesn't just rename things; it **shrinks code**, **optimizes instructions**, and **inlines functions**.
- **Rule Fragility:** Traditional tools rely on strict matching rules. If R8 changes the code structure slightly, the rules break, leading to massive false negatives.
- **Labeled Data Scarcity:** Training a deep learning model usually requires millions of "labeled" examples (knowing exactly which lib is in which app), which is hard to get in the real world.

4. How does it work? (The Methodology)

LibAttention uses a "Pre-train and Fine-tune" approach, which is common in modern AI (like ChatGPT) but new for library detection:

- **Feature Extraction (Smali):** It converts the binary code into **Smali** (intermediate code) but ignores the "volatile" parts like variable names and the actual logic inside function bodies. It only keeps the "stable" parts: **structural symbols** and **SDK types** that can't be obfuscated without breaking the app.
- **The GPT-like Encoder:** They trained a Transformer model from scratch using a huge dataset of 200 million "tokens" from the AndroZoo market. This allows the model to learn the "language" of Android code autonomously without needing human labels.
- **Data Compression:** Because app code is huge, they use an algorithm to compress the model's output into a **dense, low-dimensional vector** that is small enough for a final classification model to handle.
- **Downstream Prediction:** A final simple layer (binary classifier) takes the vectors of an app and a library and outputs a **0 or 1**—telling you if the library is inside that app.

5. Key Findings of the Paper

- **R8 Killer:** LibAttention achieved a **30% improvement in F1-score** over previous tools when tested on the modern R8 obfuscation dataset.
- **Real-World Impact:** In a scan of **10,000 real apps**, it identified **28 different vulnerable libraries** that appeared over **42,000 times**, proving that the security risk from TPLs is still massive.
- **Efficiency:** Even though it uses complex deep learning, it can process and represent code features in a way that is scalable for large-scale market analysis.

6. Impact on Project

This paper is the most "future-proof" one for my PFE:

1. **Dealing with R8:** It warns us that if we only use rule-based methods (like in the LibScout or LibD papers), our tool will fail on modern apps compiled with Android Studio 3.4+.
2. **Vocabulary Strategy:** Their idea of a "Vocabulary" of reserved SDK types is something we can definitely use to build our own feature extractor.
3. **The "AI" Advantage:** It shows that we don't need a huge labeled dataset to use AI; we can use **unlabeled data** from the app market to help our tool "learn" the structure of Android apps before we even start the detection phase.

Orlis: Obfuscation-resilient library detection for android

1. What is it?

This paper introduces **ORLIS**, a tool designed to detect third-party libraries (TPLs) even when they are hidden by very aggressive obfuscation. While other tools focus on names or simple structures, ORLIS is "obfuscation-resilient" because it focuses on the **Control Flow Graph (CFG)** and how code actually executes, rather than how it is named.

2. Who wrote it and who is it for?

- **Authors:** Researchers from **The Ohio State University**, led by Yan Wang and Atanas Rountev.
- **Audience:** It is for security analysts who need to identify libraries in apps that have been protected by professional-grade obfuscators (like DashO or Allatori), which go far beyond simple renaming.

3. Why was it created? (The Problem)

The authors argue that previous tools like LibScout or LibRadar have a "weak spot":

- **The Fragility of Signatures:** Most tools use "coarse-grained" features (like the number of classes or method descriptors). If an obfuscator merges classes or changes method parameters, these tools fail.
- **Aggressive Obfuscation:** Modern obfuscators can change the "shape" of the code logic. The authors wanted to prove that even if you change the names and the classes, the **internal logic flow** of a library is much harder to hide.
- **Library Customization:** Developers often only include pieces of a library. ORLIS was created to find these "partial" libraries with high precision.

4. How does it work? (The Methodology)

ORLIS uses a "fine-grained" approach that looks deep into the code's behavior:

- **Control Flow Graph (CFG) Hashing:** It extracts the CFG of every method. It then generates a hash based on the **structure of the graph** (the blocks and the jumps) and the **instructions** (opcodes) inside those blocks.
- **Two-Step Matching:**
    1. **Filtering (Coarse):** It first uses simple features (like the number of methods in a class) to quickly narrow down which libraries *might* be in the app.
    2. **Verification (Fine):** It then performs a detailed comparison of the CFGs. It uses a **Jaccard Similarity** score to decide if two methods are the same.
- **Handling Renaming and Merging:** Because it looks at the "flow" inside the methods, it doesn't care if a class was renamed or if two classes were merged together; the logical path of the instructions remains a strong fingerprint.

5. Key Findings of the Paper

- **High Precision:** In tests against apps obfuscated with **Allatori** and **ProGuard**, ORLIS maintained very high precision (around 90% or higher).
- **Version Sensitivity:** Because it looks at the exact instructions, it is very good at telling the difference between Version 1.1 and 1.2 of a library, even if only a few lines of code were changed.
- **Real-World Discovery:** The authors scanned 2,000 popular apps and found that many were using libraries with **expired versions** that had known security risks, which the developers were trying to hide using obfuscation.

6. Impact on Project

ORLIS gives us a very important technical lesson for our prototype:

1. **Logic is better than Names:** If we want our tool to be professional, we cannot just rely on Smali tokens or package structures. We should consider the **CFG** as a feature.
2. **Jaccard Similarity:** We can use this mathematical approach $A \cap B / A \cup B$ to handle "partial" matches when an app developer has deleted some parts of a library to save space.
3. **The Accuracy Trade-off:** The paper shows that "fine-grained" detection is slower but much more accurate. For our project, we might want to offer two modes: a "Fast Scan" (like LibRadar) and a "Deep Audit" (like ORLIS).

did’nt find the whole paper

Obfuscation-resilient detection of Android third-party libraries using multi-scale code dependency fusion

1. What is it?

This paper describes a framework called **LibMD**. Its main purpose is to generate a Software Bill of Materials (SBOM) for Android APKs. It is a very modern tool (from 2025) that doesn't just look at library names, but instead focuses on "logical dependencies" and deep code structures to identify the exact version of a library.

2. Who wrote it and who is it for?

- **Authors:** A team led by **Zhao Zhang and Senlin Luo**, who are experts in info-security.
- **Audience:** This is mostly for security auditors. It is very useful when you have an app that is heavily obfuscated with ProGuard or R8 and you need to find specific CVEs (vulnerabilities) without having the original source code.

3. Why was it created? (The Problem)

The authors noticed that current detection tools have two big weaknesses:

- **The Obfuscation Gap:** Many tools only check one thing, like method names. If the developer uses "Control Flow Flattening" or renames everything, the tool breaks.
- **The Version Gap:** It is extremely hard to tell the difference between version 1.2.1 and 1.2.2 because they are almost identical. This is a problem because a security patch might exist in one but not the other.

4. How does it work? (The Method)

The "secret" to LibMD is that it uses a **three-layered approach** to create a fingerprint:

- **Multi-Scale Extraction:** It scans the code at three different levels:
    1. **Class Scale:** Relationships between classes.
    2. **Function Scale:** How functions call each other across the app.
    3. **Basic Block Scale:** The actual logic inside the instructions.
- **Dependency Fusion:** It "fuses" these three levels. The idea is that even if a class is renamed, the way the functions call each other and the logic inside the blocks don't change.
- **Library Dependency Graph (LDG):** It creates a map of how libraries usually depend on other libraries to help narrow down the search.

- **Bayesian Probability:** I found this part interesting—the tool uses **Bayes' Theorem** to calculate a probability score. It basically asks: "Based on the evidence we see in the code, what is the mathematical chance that this is exactly Version 2.0?"

5. Key Findings of the Paper

The paper shows that by combining these different "scales" of code analysis, the tool becomes much harder to trick. Even with aggressive R8 optimizations, the "shape" of the code's dependencies remains visible. By using probability instead of just a "yes/no" rule, LibMD is much better at distinguishing between very similar library versions than older tools.

6. Impact on Project

This paper is a great conclusion for my research because:

- It confirms that we shouldn't rely on just one feature (like just hashes or just API calls). We need **"Multi-scale" evidence**.
- It gives me a high-level goal: to look for the "shape" of how code functions rather than just looking at strings or names.
- Using a probability-based approach (like the Bayesian logic) could be a very professional way to handle "uncertain" matches in our prototype.

Atvhunter: Reliable version detection of third-party libraries for vulnerability identification in android applications

1. What is it?

This paper introduces **ATVHUNTER**, a specialized system designed to detect the exact versions of third-party libraries (TPLs) used in Android apps. Its primary goal is to bridge the gap between library detection and vulnerability management by linking specific library versions to known security flaws (CVEs).

2. Who wrote it and who is it for?

- **Authors:** A group of researchers from institutions like **The Hong Kong Polytechnic University** and **Nanyang Technological University**.
- **Audience:** This is for security auditors and market maintainers. It helps them scan thousands of apps to identify which ones are "vulnerable by association" because they use an outdated or buggy library version.

3. Why was it created? (The Problem)

The authors noticed that current detection tools were not reliable enough for real-world security audits. They identified several major issues:

- **The Dependency Problem:** Many libraries include other libraries. If a tool doesn't understand these "nested" dependencies, it might give wrong results.
- **Obfuscation & Customization:** Developers often rename code or only include the small parts of a library they actually need (partial inclusion). This makes it very hard for standard tools to find a match.
- **The Version-Security Link:** Knowing a library is present isn't enough; you need to know the **exact version** to check if it contains a specific CVE. Without this, you can't accurately identify security risks.

4. How does it work? (The Methodology)

The technical core of ATVHUNTER is divided into four main parts that work together to "unmask" the libraries:

- **A. Module Decoupling (The "Split"):** Since we can't trust package names because of obfuscation, the tool uses **Class Dependency Graphs (CDG)**. It analyzes how classes "talk" to each other to group them into independent modules. This helps the tool figure out where the app code ends and the library code begins.
- **B. Two-Phase Matching:** To handle millions of potential library versions without crashing the system, it uses two steps:

    1. **Coarse-Grained (Speed):** It extracts the **Control Flow Graph (CFG)** of methods. This acts as a "skeleton." If this skeleton doesn't match what is in the database, the candidate is thrown out immediately to save time.
    2. **Fine-Grained (Precision):** For the remaining candidates, the tool looks at the **Opcodes** (the low-level instructions). It uses **Feature Weighting** to focus on the unique parts of the code rather than common "boilerplate" code.
- **C. Fuzzy Hashing:** To beat renaming or minor code changes, ATVHUNTER uses **Fuzzy Hashing (specifically a Rolling Hash)**. This allows the system to calculate a similarity percentage. It can say, "This code is 95% similar to Version 1.2," even if a few instructions were changed by the compiler or an obfuscator.
- **D. Vulnerability Mapping:** The researchers built a massive database of **1,180 CVEs** and **224 security bugs**. Once a version is identified, the system automatically flags any associated security risks, making the audit process much faster.

5. Key Findings of the Paper

The results of their study were quite eye-opening:

- **Massive Risk:** They analyzed over **60,000 apps** and found that **87.5%** of them contained at least one vulnerable library version.
- **Efficiency:** Their "Two-Phase" approach proved to be much faster than deep-learning-based tools while maintaining a high precision (over 90%).
- **New Vulnerabilities:** They actually discovered **157 previously unknown** vulnerable apps that were using "hidden" or customized library versions that other tools missed.

6. Impact on Project

This paper gives me a very clear technical path for my own prototype:

1. **Don't rely on names:** We must use **CDGs** to identify library boundaries because obfuscation is too common to ignore.
2. **Use Feature Weighting:** When comparing versions, we should focus on the parts of the code that are unique to that version.
3. **Automate the CVE check:** A list of libraries is just data, but a list of **CVEs** is a result. We should definitely try to map our findings to a vulnerability database to add more value to our SBOM.

Version-level Third-Party Library Detection in Android Applications via Class Structural Similarity (2025)

1. What is it?

This paper introduces a tool called **SAD** (Structural Analysis-based Detection). Its main specialty is **version-level identification**. While other tools might tell you "This is OkHttp," SAD is designed to tell you exactly if it is **v3.12.1** or **v3.12.0**, which is very important for checking security patches.

2. Who is it for?

- **Primary Audience:** Security researchers and people working on **SCA** (Software Composition Analysis).
- **Audience:** It solves the biggest headache in automation: **False Positives**. It stops the system from reporting five different versions of the same library, which makes the CVE matching process much cleaner.

3. Why was it created? (The Problem)

The authors identified two major failures in existing tools (like LibScan):

- **Obfuscation Vulnerability:** Most tools look at code structures that are easily destroyed by **ProGuard** or **R8**.
- **Version "Blindness":** Because version 1.1 and 1.2 are 99% similar, most tools get confused and report the wrong one. SAD was built specifically to notice the tiny differences between these "adjacent" versions.

4. How does it work? (The Methodology)

SAD uses a unique three-step pipeline:

- **A. Preprocessing: The Class Dependency Graph (CDG)**
  Instead of trusting names, it builds a graph where each node is a Class. It uses a technique called **"neighbor feature aggregation"** (inspired by Graph Neural Networks).
    - *Insight:* Even if the class Logger is renamed to a, it still talks to the same "neighbor" classes. SAD captures this "social network" of the code to identify it.
- **B. Class Matching: Stateful vs. Stateless**
  This is the most innovative part. SAD separates classes into two types:
    - **Stateful Classes:** These are classes that have "fields" (variables). SAD tracks how the methods read or write to these fields (the **Field Operation Sequence**). This logical flow is very hard for an obfuscator to hide.
    - **Stateless Classes:** These classes have no fields (just logic). For these, SAD matches them using **Opcode** (instruction) similarity.

- **C. Tie-Breaking with Literal Features**
  If two versions are extremely similar and have the same score, SAD uses a "tie-breaker." It looks for **Literal Features**—specific strings or constant numbers hardcoded in the app—to make the final decision.

5. Key Findings of the Paper

- **Superior Accuracy:** SAD achieved a **Precision of 97%** and a **Recall of 98%** at the library level.
- **Version-Level King:** In the specific task of distinguishing versions, it achieved an F1-score of **84.82%**, which is **10% higher** than the next best tool (LIBLOOM).
- **R8 Proof:** The authors proved that their "CDG" method is much more stable than instruction-based hashes when an app is compiled with modern obfuscators.

6. Impact on Project

The findings from the SAD paper provide three clear directives for the design of a modern SBOM tool:

1. **Stateful vs Stateless Distinction:** The paper demonstrates that separating classes based on whether they contain fields (variables) or just methods significantly improves accuracy. This logic provides a concrete method to handle classes differently during the matching phase, making the tool more robust against obfuscation.
2. **The Necessity of Tie-Breakers:** The research highlights that structural analysis alone is sometimes insufficient for distinguishing adjacent versions (e.g., v1.1 vs. v1.2). The use of "Literal Features" (like error strings or constants) serves as a necessary secondary filter to resolve these close matches.
3. **Graph-Based Resilience:** The results confirm that for modern Android apps compiled with R8, relying on simple package names is obsolete. A reliable detection system requires the use of **Class Dependency Graphs (CDG)** to identify libraries based on their internal relationships rather than their file names.

A systematic assessment on android third-party library detection tools

1. What is it?

This paper is a massive **"benchmarking study"** . Instead of creating one new tool, the authors took all the major existing tools—**LibRadar, LibScout, LibID, LibPecker, and ORLIS**—and tested them side-by-side to see which one is actually the best in 2020/2021.

2. Who wrote it and who is it for?

- **Authors:** A large team from **The Hong Kong Polytechnic University, Monash University, and Nanyang Technological University**.
- **Audience:** It is a "Buyer's Guide" for developers and researchers. It tells us which tool to choose based on whether we need speed, accuracy, or obfuscation resilience.

3. Why was it created? (The Problem)

The authors noticed huge inconsistencies in the field:

- **The "Multidex" Blind Spot:** They suspected many tools were failing on modern, large apps because they couldn't handle "Multidex" (apps split into multiple `.dex` files).
- **Format Incompatibility:** Many older tools were built for `.jar` files and broke when trying to scan modern Android `.aar` libraries.
- **Unfair Comparisons:** Previous papers used different datasets, making it impossible to know which tool was truly superior.

4. How does it work? (The Methodology)

They built a unified benchmark to test six criteria:

1. **Module Decoupling:** Can the tool separate library code from app code?
2. **Effectiveness:** Precision and Recall.
3. **Efficiency:** Scanning speed (seconds per app).
4. **Version Identification:** Distinguishing v1.0 from v1.1.
5. **Obfuscation Resilience:** Testing against **ProGuard, Allatori, and DashO**.
6. **Ease of Use:** How easy is it to install and run?

5. Key Findings

- **Speed King: LibRadar** is the fastest (~5 seconds/app) but struggles with specific versions.
- **Accuracy King: LibScout** is the most reliable for finding specific versions but is slower.
- **Obfuscation King: ORLIS** handles obfuscation best because it uses Control Flow Graphs (CFG) instead of names.

6. Impact on Project

This paper validates the "Hybrid" approach. It proves that no single tool is perfect, so we should combine the **speed of LibRadar** (for a quick scan) with the **graph analysis of ORLIS** (for deep verification).

7. Technical Implementation: How they Patched the Tools

The authors didn't just test the tools; they found critical bugs in them and wrote code to fix them. Here is how they did it:

- **A. The "Multidex" Patch (Fixing the 50% Blind Spot)**
  - **The Bug:** Tools like LibRadar and LibID had hardcoded logic that only looked for a file named `classes.dex`. In large apps (like Facebook or Uber), the code is split into `classes2.dex`, `classes3.dex`, etc. The tools were ignoring these extra files, missing half the libraries.
  - **The Fix:** The authors modified the source code to replace the single file check with a **loop**.
    - *Step 1:* The patch scans the APK for **all** files ending in `.dex`.
    - *Step 2:* It merges the class hierarchies from all these files into a single "ClassPool" in memory.
    - *Step 3:* The detection algorithm is then run on this unified pool, ensuring 100% of the code is visible.
- **B. The ".aar" Format Patch**
  - **The Bug:** Older tools expected libraries to be in `.jar` format. Modern Android libraries are distributed as `.aar` (Android Archive), which is a zip file containing a `classes.jar` plus resources (images, XML). When tools tried to read an `.aar` as a `.jar`, they crashed.
  - **The Fix:** They added a **Pre-processing Script**.
    - Before the tool starts, the script detects if the input is an `.aar`.
    - If yes, it automatically unzips the archive, extracts the internal `classes.jar`, and feeds *that* clean JAR file into the detection tool. This made tools like LibPecker compatible with modern libraries instantly.
- **C. The "EntryPoint" Patch (for Obfuscation)**
  - **The Bug:** Some tools assumed that a library always starts with a unique package name (like `com.squareup.okhttp`). When obfuscation renamed this to `a.b.c`, the tools couldn't even find where to start looking.
  - **The Fix:** They patched the "Root Detection" logic. Instead of looking for a specific string name, they modified the tools to look for **Class Structure Roots**—classes that have no dependencies on other classes in the same package. This allows the tool to find the "start" of a library even if the name is completely scrambled.

Empirical Study of Third-Party Library Detection in iOS and Android Applications (2024)

1. What is it?

This is a Master's Thesis submitted at the **Technical University of Madrid (UPM)**. It serves two main purposes:

1. It provides a complete **"State of the Art"** review of library detection history (2014–2024).
2. It attempts to bridge the gap between Android and iOS detection by benchmarking established Android tools against the very limited options available for iOS.

2. Who is it for?

- **Authors:** Pablo Marchand Ramirez (Student) and Alessandra Gorla (Tutor/Researcher).
- **Audience:** This is critical for researchers who need to understand the **"technological gap"** between Android and iOS. It explains why we can easily analyze Android apps but struggle to see inside iPhone apps.

3. Why was it created? (Background & The Problem)

Covering ToC Sections 2.1, 2.2, 3.1

The author identifies that while mobile development relies heavily on Third-Party Libraries (TPLs), the security tools for them are unbalanced.

- **The iOS Environment (2.1):**
  - **Development:** iOS apps are built with Xcode using Swift or Objective-C.
  - **Structure of Binaries (2.1.2):** iOS uses **Mach-O** binaries. The biggest problem is **"Static Linking"**—the library code is compiled *into* the main app executable. There are no clear file boundaries.
  - **Security (2.1.3):** iOS uses **ASLR** (Address Space Layout Randomization) and strict **Code Signing**, making reverse engineering very difficult compared to Android.
- **The Android Environment (2.2):**
  - **Structure of Binaries (2.2.2):** Android uses **DEX** (Dalvik Executable) files. Even after compilation, these files keep their class structures and package hierarchies (e.g., `com.squareup.okhttp`), which makes detection much easier.
- **The General Problem (3.1):** Both ecosystems use massive amounts of libraries (3.1.1), but while Android libraries are often separate `.jar` or `.aar` files (3.1.3), iOS libraries are invisible inside the binary (3.1.2).

4. How does it work? (State of the Art & Methodology)

Covering ToC Sections 3.2, 3.3, 3.4, 4.1, 4.2.1, 4.3.1

The author first categorized all existing tools (Literature Review) and then designed an experiment.

A. Taxonomy of Techniques (3.2):

He classifies detection methods into: **Hashing** (fast/brittle), **Feature-based** (strings/constants), and **Dependency Graphs** (slow/accurate).

B. The "State of the Art" Tool Review:

- **iOS Tools (3.3):** He reviews the very few attempts made for iOS.
    - **CRIOS (3.3.1) & iLibScope (3.3.2):** Early academic attempts to find libraries using static analysis, but they are not publicly available.
    - **LibKit (3.3.3):** A newer tool, but limited in scope.
    - **Selected for Test:** He chose **BliS** (not listed in the header but used in the experiment) and analyzed **Class-dump** (a utility to reverse header files).
- **Android Tools (3.4):** He provides a detailed history of Android tools.
    - **AdDetect (3.4.1):** Only finds ad libraries.
    - **LibRadar (3.4.2):** The standard for hashing (Package Structure).
    - **LibScout (3.4.3):** The standard for Profile Matching (Merkle Trees).
    - **ORLIS (3.4.4):** Uses Control Flow Graphs (CFG) for obfuscation resilience.
    - **LibID (3.4.5):** Relies on package naming conventions (found to be fragile).
    - **LIBLOOM (3.4.6) & LibScan (3.4.7):** Newer tools using fuzzy matching.

C. Experimental Setup:

- **Android (4.2.1):** He used **F-Droid** to build a "Ground Truth" dataset because open-source apps allow 100% verification of libraries.
- **iOS (4.3.2):** Since no "F-Droid for iOS" exists, he had to **manually** download and verify a set of iOS apps to create a Ground Truth.

5. Key Findings (The Results)

Covering ToC Sections 4.2, 4.3, 4.4, 5.1, 5.2, 5.3

Android Results (4.2):

- **Effectiveness (4.2.2): LibScout** is the most accurate. **LibID** failed on modern apps because it cannot handle the new **R8** compiler.
- **Efficiency (4.2.3): LibRadar** is the fastest (seconds). **ORLIS** is the slowest and heaviest—it consumed so much RAM (16GB+) that it crashed on large apps.
- **Version Identification (4.2.4):** Only LibScout could reliably tell v1.0 from v1.1.
- **Obfuscation (4.2.5):** ProGuard and R8 significantly hurt detection, but graph-based tools survived better than package-based tools.

iOS Results (4.3):

- **Efficiency (4.3.4):** The tool **BliS** is fast because it only scans strings.
- **The Failure (4.3.5):** iOS detection has **zero** capability for version identification.
- **Obfuscation:** When "Symbol Stripping" is enabled (removing debug names), iOS detection drops to **0%**.

User Experience (4.4):

Android tools are generally usable (though ORLIS is hard to install). iOS tools are mostly "abandonware" or require complex manual setups.

Comparison of Usage (Chapter 5):

- **Volume (5.1):** Android apps use *more* TPLs on average than iOS apps.
- **Types (5.2):** Android relies on Ads/Utilities. iOS relies on UI/Networking frameworks.
- **Vulnerabilities (5.3):** The study found far more CVEs (vulnerabilities) in Android apps. However, the author notes this might just be because iOS detection tools are too blind to find the bugs.

6. Impact on Project (Conclusion)

Covering ToC Chapter 6

- **Objectives Review (6.1):** The thesis confirms that Android detection is mature, while iOS detection is 10 years behind.
- **Takeaway:**

1. **Justification for Ignoring iOS:** It proves that including iOS is out of scope. The thesis proves that without "Symbol Tables," detection is impossible using current open-source methods.
2. **Architecture Decision (The Hybrid Logic):**
    - The experiments prove that **LibRadar** is too inaccurate alone, and **ORLIS** is too slow/heavy alone.
    - **the Solution:** use a **Pipeline**. Run LibRadar first (5 seconds). If it finds a match, run a "Lite" version of LibScout or SAD to verify it. This avoids the 20-minute wait time of ORLIS.
3. **The "R8" Requirement:** The failure of LibID proves that I cannot rely on "Package Names." the tool **must** use "Class Structural Features" (like checking the number of methods/fields) because that is the only thing R8 cannot hide.
4. **Hardware Optimization:** The crash reports for ORLIS warn me that I must implement **Timeouts** and **Memory Limits** in my backend, or one large APK will crash my entire server.